سیستم تشخیص خودروهای ترافیکی :

خلاصه ی مسیری که در کل رفتیم :

توی این پروژه، ما یه سیستم برای تشخیص خودروهای ترافیکی از تصاویر دوربین مدار بسته ساختیم. دیتاست ما ۸ کلاس مختلف داشت: آمبولانس، اتوبوس، کامیون، کامیونت، مینی‌بوس، سواری، تاکسی و وانت.

کار رو با پاک‌سازی دیتا شروع کردیم. با استفاده از هش MD5 و هش ادراکی (average hash)، تصاویر تکراری رو پیدا کردیم. بعد با استفاده از embedding مدلResNet ، تصاویری که احتمال می‌دادیم اشتباه برچسب خورده باشن رو پیدا کردیم و بررسی کردیم. در نهایت یه split تمیز از train و validation ساختیم و testرو فریز کردیم.

بعد از پاک‌سازی، یه مدل CNN ساده به‌عنوان baseline ساختیم و آموزش دادیم. بعد کلی آزمایش (ablation) انجام دادیمaugmentation :، dropout، pooling، weight decayو .scheduler نتیجه این بود که هیچ‌کدوم از این‌ها

کمکی نکردن — یعنی مدل baseline از قبل خوب بود و regularization اضافه فقط ضرر می‌زد.

بعد سراغ imbalance رفتیم. یه دیتای نامتعادل مصنوعی ساختیم (بعضی کلاس‌ها رو کم کردیم) و با یه sampler متعادل‌کننده (BalancedBatchSampler) تونستیم recall کلاس‌های نادر رو از ۰ به ۰,۷۸ برسونیم.

بعد CE و BCE رو با هم مقایسه کردیم. نتیجه جالب بود BCE :توی lr=1e-3 بدتر بود، ولی با lr=3e-3 از CE بهتر شد. این نشون داد که مقایسه‌ی عادلانه نیاز به tuningبرابر داره.

بخش مهم پروژه، Transfer Learningبود. یه ResNet-18 از پیش آموزش‌دیده روی ImageNet رو آوردیم و به سه روش تست کردیم feature extraction :)فقط لایه آخر(، fine-tuning(لایه‌های آخر + لایه آخر)، و from scratch (از

صفر). نتیجه این شد که fine-tuning بهترین عملکرد رو داشت = macro-F1 : 0.932.

در نهایت، تحلیل‌های تکمیلی انجام دادیمconfusion matrix .: ، pair confusion، تحلیل تصاویر اشتباه، و تحلیل کلاس .(neysan) unseen این تحلیل‌ها نشون داد که مدل روی کلاس «وانت» ضعیف‌تره و اعتماد بیش از حد (overconfidence)داره.

خروجی نهایی: یه مدل ResNet-18 با fine-tuning که به macro-F1 = 0.932 رسید، به همراه یه تابع predict که خروجی JSON می‌ده و یه threshold برای needs_review.

مقدمه :

تشخیص خودروها از تصاویر ترافیکی یه مسئله‌ی مهم توی دنیای امروزه. ازش توی نظارت ترافیک، مدیریت شهری، شمارش خودروها، و حتی سیستم‌های خودران استفاده می‌شه. ولی این کار چالش‌های خاص خودش رو داره: تصاویر ممکنه تار باشن، خودروها از زوایای مختلف دیده بشن، بعضی کلاس‌ها شبیه هم باشن (مثل کامیون و

کامیونت و یا آمبولانس و اتوبوس)، و بعضی کلاس‌ها نمونه‌ی کم داشته باشن (مثل آمبولانس).

روش‌های سنتی برای تشخیص خودرو معمولاً بر اساس ویژگی‌های دستی) مثل SIFT یا (HOG بودن، ولی این روش‌ها توی تصاویر واقعی و متنوع خوب کار نمی‌کنن. با پیشرفت یادگیری عمیق، شبکه‌های عصبی کانولوشنی (CNN) تونستن عملکرد خیلی بهتری ارائه بدن.

ولی یه مشکل مهم وجود داره: آموزش CNN از صفر به دیتای زیاد نیاز داره. توی پروژه‌های واقعی، معمولاً دیتای برچسب‌خورده کمه. اینجاست که Transfer Learningوارد می‌شه.

Transfer Learningیعنی استفاده از یه مدل که از قبل روی یه دیتاست بزرگ مثل ImageNet با ۱,۲ میلیون تصویر و ۱۰۰۰ کلا آموزش دیده. این مدل ویژگی‌های عمومی مثل لبه، بافت، شکل و اشیاء رو یاد گرفته. ما می‌تونیم این دانش رو بگیریم و روی دیتای خودمون fine-tune کنیم.

توی این پروژه، ما سه استراتژی مختلف رو تست کردیم:

۱ .آموزش از صفر CNNBaseline) ساده(

۲ .آموزش از صفر با معماریResNet

۳ .استفاده از ResNet از پیش آموزش‌دیده) با دو روش feature extraction :و fine-tuning)

هدف این بود که بفهمیم کدوم استراتژی برای دیتای ما بهتر جواب می‌ده و چرا.

۱ .دیتاست و پاک‌سازی

دیتاست ما شامل تصاویر برش‌خورده‌ی خودروها از فریم‌های دوربین ترافیکی بود.

دیتاست سه بخش داشت:

Train: ۸ کلاس، حدود ۱۹۲۸ تصویر

unclean : 9 کلاس (تقریبا 250 عکس بیشتر نیسان)

Test: که در ابتدا نیست و بعداً توسط استاد داده می‌شه

چرا پاک‌سازی لازم بود؟

توی دیتای خام، چند مشکل وجود داشت:

۱ .تصاویر تکراری (همون عکس توی دو کلاس مختلف)

۲ .اشتباه برچسب‌گذاری (مثلاً نعش‌کش به‌عنوان آمبولانس)

۳ .تصاویری که به هیچ کلاسی تعلق نداشتن

۴ .کلاس unseen (neysan) که نباید توی آموزش باشه

روش های پاک‌سازی:

اول با هش MD5 تصاویر کاملاً یکسان رو پیدا کردیم. ۲۴ گروه تکراری پیدا شد که یکیشون تناقض برچسب داشت) یه نیسان توی train با برچسب vanet و توی unclean با برچسبneysan تصمیم گرفتیم نسخه‌ی train رو حذف کنیم و نسخه‌ی neysan رو نگه داریم.

بعد با هش ادراکی (average hash) تصاویر مشابه رو پیدا کردیم. ۴۶ گروه کاندید پیدا شد، ولی بعد از بررسی چشمی، فهمیدیم بیشترشون false positive هستن (مثلاً دو تاکسی با رنگ متفاوت یا یه تاکسی و یه سواری از یه زاویه). پس هیچ‌کدوم رو حذف نکردیم. بعد با embedding ResNet تصاویری که احتمال می‌دادیم اشتباه برچسب خورده باشن رو پیدا کردیم. روش این بود: برای هر تصویر یه بردار ۵۱۲ بعدی

گرفتیم، centroidهر کلاس رو حساب کردیم، و فاصله‌ی هر تصویر از centroid کلاس خودش رو حساب کردیم. ۱۵ تصویر دورتر از همه رو بررسی کردیم. نتیجه: ۳ تصویر به کلاس درست منتقل شد، ۳ تصویر حذف شد (چون به هیچ کلاسی تعلق نداشتن — مثلاً ماشین پلیس انتظامی یا راهنمایی رانندگی)، و بقیه نگه داشته شد.

خروجی نهایی:

train: ۱۵۳۹ تصویر

val: ۳۸۵ تصویر

test: ۴۰۰ تصویر (فریز شده)

چالش‌ها:

۱ .هش ادراکی خیلی false positive داشت — چون فقط الگوی کلی روشنایی رو می‌بینه، نه هویت خودرو.

۲ .بعضی تصاویر واقعاً مبهم بودن — مثلاً آمبولانس‌هایی که روی شاسی مزدا وانت ساخته شدن و از نظر بصری شبیه وانت هستن.

3. کلاس‌های سخت minibus/autobus وkamyun/kamyunet رو به‌صورت دسته‌جمعی تغییر ندادیم فقط موارد واضح را تغییر دادیم .

۲ .مدل پایه(CNNBaseline)

برای شروع، یه مدل CNN ساده ساختیم که اسمش رو گذاشتیم .CNNBaseline

این مدل ۳ تا بلوک کانولوشنی داشت:

بلوک ۱(2)MaxPool → ReLU → (3→32) Conv3×3 :

بلوک ۲(2)MaxPool → ReLU → (32→64) Conv3×3 :

بلوک ۳(2)MaxPool → ReLU → (64→128) Conv3×3 :

بعدGlobal Average Pooling :

آخر(8 → 128)Linear → (0)Dropout :

تعداد پارامترها: ۹۴،۲۸۰

تنظیمات آموزش:

ورودی: ۱۲۸×۱۲۸

نرمال‌سازیImageNet mean/std :

Optimizer: AdamW با lr=1e-3

Loss: CrossEntropy

Batch size: 32

Epochs: 30

بدون augmentation

نتیجه:

macro_F1 : ۰,۶۵۶

Accuracy : ۰,۶۵۹

best_epoch: ۲۸

gap (train-val) : ۰,۰۹۴

ضعیف‌ترین کلاس kamyunet F1 ۰,۳۰

تحلیل:

مدل پایه نسبتاً خوب کار کرد .(F1 = 0.656) ولی دو تا مشکل داشت:

1 .کلاس kamyunet خیلی ضعیف بود — (F1 = 0.30) یعنی مدل نمی‌تونست کامیون و کامیونت رو از هم تشخیص بده.

۲ gap .کوچیک بود (0.094) — یعنی مدل overfit نمی‌کرد، پس به regularizationنیاز نداشت.

و در مسیر پروژه این baseline مرجع مقایسه‌ی همه‌ی آزمایش‌های بعدی شد.

۳ .آزمایش‌ها(Ablations)

بعد ازbaseline ، یه سری آزمایش انجام دادیم که هر بار فقط یه چیز رو عوض می‌کردیم. هدف این بود که بفهمیم کدوم قطعه چقدر تأثیر داره

Augmentation:

Augmentationیعنی تغییر مصنوعی تصاویر (چرخش، برش، تغییر رنگ) که مدل بهتر generalize کنه. ما این تنظیمات رو تست کردیم:

transforms.RandomResizedCrop(128, scale=(0.8, 1.0))

transforms.RandomHorizontalFlip()

transforms.RandomRotation(10)

transforms.ColorJitter(brightness=0.3, contrast=0.3)

نتیجه F1 :افتاد از ۰,۶۵۶ به ۰,۶۲۰ – ۰,۰۳۶.

تحلیل augmentation :کمکی نکرد. چرا؟ چون دیتای ما کوچیک بود (۱۹۲۸ تصویر) و augmentation سخت‌ترش می‌کرد. همچنین مدل از قبل overfit نمی‌کرد.

Dropout:

Dropoutیعنی خاموش کردن تصادفی بعضی نورون‌ها در طول آموزش. دو مقدار تست کردیم: ۰,۳ و ۰,۵.

نتیجه:

dropout=0.3 → F1 = ۰,۶۰۸ ۰,۰۴۸

dropout=0.5 → F1 = ۰,۶۲۱ ۰,۰۳۵

تحلیل: هر دو بدتر شدن gapخیلی کوچیک شد (۰,۰۱۶ و −۰,۰۲۵) که نشون می‌ده مدل underfit شد. یعنی regularization اضافه ضرر زد.

Pooling (Max vs Avg) :

MaxPoolبزرگ‌ترین عدد رو نگه می‌داره، AvgPoolمیانگین رو.

نتیجه = F1 → AvgPool :۰,۵۹۷ ۰,۰۵۹ که بدترین نتیجه میشه..

تحلیل MaxPool :بهتره چون لبه‌ها و ویژگی‌های متمایز رو حفظ می‌کنه .

AvgPoolاینا رو نرم می‌کنه و برای تشخیص کامیون vs کامیونت مضرره.

Weight Decay:

Weight decayیه نوع regularization هست که وزن‌ها رو کوچیک نگه می‌داره.

نتیجه = F1 :۰,۶۳۰ ۰,۰۲۶تقریباً بی‌اثر.

تحلیل: ۱ e-4خیلی کوچیک بود که تأثیر داشته باشه.

Scheduler (ReduceLROnPlateau):

Schedulerیعنی کم کردن learning rate وقتی مدل دیگه بهتر نمی‌شه.

نتیجه = F1 :۰,۶۴۹ ۰,۰۰۷نزدیکbaseline ، ولی نه بهتر.

تحلیل: بهترین آزمایش غیر baseline-بود. بهترینepoch ، epoch۳۰ (آخرین)

بود — یعنی مدل هنوز داشت بهتر می‌شد.

جمع‌بندی آزمایش‌ها:

baseline → ۰٫۶۵۶

scheduler → ۰٫۶۴۹ (−۰٫۰۰۷)

weight decay → ۰٫۶۳۰ (−۰٫۰۲۶)

dropout 0.5 → ۰٫۶۲۱ (−۰٫۰۳۵)

augmentation → ۰٫۶۲۰ (−۰٫۰۳۶)

dropout 0.3 → ۰٫۶۰۸ (−۰٫۰۴۸)

avg pool → ۰٫۵۹۷ (−۰٫۰۵۹)

درس بزرگ: هیچ‌کدوم از آزمایش‌ها بهتر از baseline نشدن. این یعنی مدل baseline از قبل well-regularized بود و مشکل اصلی ظرفیت مدل بود، نه regularization.

۴) Imbalance .نامتعادل بودن دیتا : توی دیتای واقعی، معمولاً بعضی کلاس‌ها نمونه‌ی بیشتری دارن و بعضی کمتر. ما یه دیتای نامتعادل مصنوعی ساختیم:

آمبولانس: ٪۲۰ (۳۵ تصویر)

مینی‌بوس: ٪۳۰ (۵۴ تصویر)

بقیه: ٪۱۰۰ (~۱۹۵ تصویر)

روش کار:

دو تا آزمایش انجام دادیم:

۱ .بدون تکنیک :(imb_standard) همون آموزش معمولی

۲ .با :(imb_balanced) BalancedBatchSampler هر batch دقیقاً ۴ نمونه از هر کلاس داشت

نتیجه:

imb_standard: F1 = ۰,۴۰۵ ambulance recall = ۰,۰۳۷ minibus recall = ۰,۰۰۰

imb_balanced: F1 = ۰,۵۳۴ambulance recall = ،۰,۷۷۸minibus recall = ۰,۸۲۲

minibus recall +۰,۸۲۲، ambulance recall +۰,۷۴۱، F1 تفاوت: +۰,۱۲۹

تحلیل:

نتیجه شگفت‌انگیز بود! بدون sampler ، مدل کلاس‌های نادر رو کاملاً نادیده می‌گرفت (recall = ۰,۰۳ و ۰,۰۰ .(با sampler ، هر دو کلاس نجات پیدا کردن.

ولی یه trade-off هم داشت: کلاس‌های پرنمونه مثل vanet و kamyunet کمی افت کردن چون sampler به اون‌ها کمتر توجه می‌کرد.

درسی که گرفتیم این بود که : وقتی دیتا نامتعادله، حتماً از sampler یا class weights استفاده کن.

۵ .مقایسه CrossEntropy وBCE

دو تا تابع loss رایج رو با هم مقایسه کردیم:

CrossEntropy: برای classification تک‌برچسبی (هر تصویر یه کلاس)

BCEWithLogitsLoss: برای classification چندبرچسبی (هر تصویر می‌تونه چند کلاس داشته باشه)

تفاوت اصلی:

CE: softmax → احتمال‌ها جمعشون ۱ می‌شه → یکی برنده

BCE: ۸ تا sigmoid مستقل → جمعشون ≠ ۱ → چند تا می‌تونن بله باشن

نتیجه) اولیه، :(lr=1e-3

CE → F1 = ۰,۵۶۵

BCE → F1 = ۰,۵۲۵

تفاوت: −۰,۰۴۰

نتیجه (tuning برابر، lr=3e-3):

CE → F1 = ۰,۵۹۶

BCE → F1 = ۰,۶۳۱

تفاوت: +۰,۰۳۵

تحلیل:

نتیجه جالب بود! توی lr=1e-3 ، CEبهتر بود. ولی وقتی هر دو رو با lr=3e-3 تست کردیم، BCEبرنده شد.

چرا؟ BCE سیگنال یاددهی ضعیف‌تره (۷ تا صفر و ۱ تا یک)، پس به lr بالاتر نیاز داره. با lr مناسب، بهتر generalize می‌کنه.

درس: مقایسه‌ی عادلانه نیاز به tuning برابر داره. اگه فقط lr=1e-3 رو تست می‌کردیم، نتیجه‌ی اشتباه می‌گرفتیم.

۶. Transfer Learning با ResNet-18.

این مهم‌ترین بخش پروژه بود. سه تا استراتژی رو تست کردیم:

۶,۱ .آموزش از صفر با CNNBaseline مرجع

F1 = ۰,۵۶۵ (قبلاً دیدیم).

۶,۲ .آموزش از صفر باResNet-18

یه ResNet-18 بدونpretrained ، مخصوص تصاویر کوچیک ۳۲×۳۲

conv1رو عوض کردیم kernel=3, stride=1 :

maxpoolرو حذف کردیم

نتیجه: F1 = ۰,۸۲۷ (+۰,۲۶۲ نسبت به CNNBaseline)

تحلیل: معماری ResNet خودش ۰,۲۶ بهبود داد. یعنی skip connections و BatchNormو عمق بیشتر کمک می‌کنن، حتی بدون pretrained.

Extraction ResNet pretrained، backboneفریز:

یه ResNet-18 از پیش آموزش‌دیده روی ImageNet رو آوردیم. همه‌ی لایه‌های backboneرو فریز کردیم و فقط لایه‌ی آخر (fc) رو آموزش دادیم.

تعداد پارامتر آموزش‌پذیر: فقط ۴،۱۰۴ (از ۱۱,۲ میلیون)

نتیجه: F1 = ۰,۸۹۸ (+۰,۳۳۳ نسبت به CNNBaseline)

تحلیل: با فقط ۴ هزار پارامتر آموزش‌پذیر، به F1 = 0.898 رسیدیم. این نشون می‌ده ویژگی‌های ImageNet فوق‌العاده قوی‌ان.

layer4 + fc Fine-tuning ResNet pretrained

همونResNet pretrained ، ولی این بار layer4 رو هم باز کردیم و با learning rateکوچیک‌تر آموزش دادیم.

تعداد پارامتر آموزش‌پذیر: ۸,۴ میلیون

LR groups:

layer4: ۱e-4 (کوچیک، چون pretrained)

fc: ۱e-3 (بزرگ، چون از صفر)

نتیجه: F1 = ۰,۹۳۲ (+۰,۳۶۷ نسبت به CNNBaseline)

تحلیل: بهترین مدل fine-tuning !باعث شد layer4 خودش رو با دیتای ما سازگار کنه. مهم‌ترین بهبود kamyunet :از F1 = ۰,۴۲ به ۰,۹۲ رسید.

جمع‌بندی:Transfer Learning

CNNBaseline (از صفر) → F1 = ۱۱,۲ ،۰,۵۶۵M پارامتر

ResNet از صفر → F1 = ۱۱,۲ ،۲۷ ۰,۸M پارامتر

ResNet Feature Extraction → F1= ۴ ،۰,۸۹۸K پارامتر

ResNet Fine-tuning → F1 = ۸,۴ ،۰,۹۳۲M پارامتر

درس: سه تا فاکتور مؤثر بودن:

۱. معماری ResNet: +۰,۲۶۲

۲. Pretrained: +۰,۰۷۱

۳. Fine-tuning: +۰,۰۳۴

۷ .نتایج نهایی و تحلیل‌ها

CNN baseline → Macro P: ،۰,۶۱۲Macro R: ،۰,۶۰۰Macro F1: ۰,۵۸۸

Balanced batches → Macro P: ،۰,۵۳۲Macro R: ،۰,۵۴۰Macro F1: ۰,۵۳۴

CE vs BCE → Macro P: ،۰,۴۹۰Macro R: ،۰,۵۳۸Macro F1: ۰,۵۰۱

Best regularized → Macro P: ،۰,۴۷۸Macro R: ،۰,۵۱۶Macro F1: ۰,۴۸۴

ResNet feature extraction → Macro P: ۰,۹۰۲Macro R: ۰,۸۹۵ Macro F1: ۰,۸۹۸

ResNet fine-tuning → Macro P: ۰,۹۳۶Macro R: ۰,۹۳۰Macro F1: ۰,۹۳۲

Confusion Matrix:

Confusion matrixرو برای مدل نهایی (ResNet-FT) ساختیم. دو تا نسخه : count-based(تعداد خام) و row-normalized درصد

یافته‌ها:

kamyun ↔ kamyunet (کامیون vs کامیونت) بیشتر خطاها بین

taxi → savari (تاکسی vs سواری)

minibus/ambulance → autobus

Pair Confusion:

جفت‌های کلاس‌ها رو رتبه‌بندی کردیم:

savari ↔ vanet → ۰,۲۳۵

kamyun ↔ kamyunet → ۰,۱۱۸

ambulance ↔ vanet → ۰,۰۵۶

بقیه ← کمتر

تحلیل: بالاترین امتیاز فقط ۰,۲۳۵ بود — یعنی حتی بدترین جفت هم کم قاطی می‌شه.

تصمیم Merge ادغام :

پروژه گفته بود: اگه دو کلاس خیلی قاطی می‌شن + دلیل دامنه‌ای، ادغامشون کن.

ما تست کردیم minibus+autobus :و kamyun+kamyunet رو merge کنیم.

نتیجه: F1 از ۰,۹۳۲ به ۰,۹۳۷ (فقط +۰,۰۰۵).

تصمیم merge : تصمیم گرفتم اینکارو نکنم چون تغییر زیادی قائل نشده.

دلیل:

۱ .بهبود ناچیز ۰,۰۰۵

۲ .تفکیک کامیون/کامیونت توی ترافیک مهمه

۳ .خطاها کم‌ان

:تصاویر اشتباه

۱۲ تصویر با بالاترین confidence که اشتباه پیش‌بینی شده بودن رو بررسی کردیم:

kamyun ↔ kamyunet → ۴

taxi → savari → ۳

minibus/ambulance → autobus → ۲

بقیه ⟶ ۳

نکته‌ی مهم: همه‌ی این ۱۲ تصویر confidence بالای ۰,۸۵ داشتن! یعنی مدل overconfidentـه — حتی وقتی اشتباه می‌کنه، مطمئنه.

تحلیل کلاس Unseen (neysan):

یه کلاس unseen به اسم neysan (نیسان) داشتیم که توی آموزش نبود. ۲۱۰ تصویر ازش رو تست کردیم:

Mean confidence: ۰٫۸۰۳

Median confidence: ۰٫۸۵۰

پیش‌بینی شد vanet → ۶۸٫۶٪

تحلیل: مدل روی unseen هم overconfident ـه. باید بگه «نمی‌شناسم»، ولی می‌گه «vanet» با confidence بالا. این نشون می‌ده که softmax به تنهایی برای تشخیص unseen کافی نیست.

انتخاب Threshold برای needs_review:

برای تصمیم‌گیری درباره‌ی اینکه کِی از کاربر بخوایم بازبینی کنه، thresholdانتخاب کردیم. روی validation تست کردیم:

Threshold ۰٫۵۰ → coverage ،۹۹٫۵٪accuracy ،۰٫۹۴۵needs_review ۰٫۵٪

Threshold ۰٫۷۰ → coverage ،۹۶٫۹٪accuracy ،۰٫۹۵۴needs_review ۳٫۱٪

Threshold ۰٫۹۰ → coverage ،۹۲٫۴٪accuracy ،۰٫۹۷۴needs_review ۷٫۶٪

انتخاب threshold = ۰٫۷۰:

دلیل coverage :بالا ۹۶٫۹ accuracyخوب (۹۵٫۴٪)، و needs_review کم ۳٫۱٪

در نتیجه :

توی این پروژه، ما یه سیستم تشخیص خودروهای ترافیکی ساختیم و از مرحله‌ی خام تا مدل نهایی رو طی کردیم.

کاری که کردیم چی بود ؟ :

۱ .دیتا رو پاک‌سازی کردیم: با هشMD5 ، هش ادراکی، وembedding ResNet ، تصاویر تکراری و اشتباه‌برچسب رو پیدا کردیم. یه split تمیز ساختیم و test رو فریز کردیم.

۲ .یه مدل پایه ساختیم CNNBaseline :با = F1 ۰,۵۸۸.

۳ .آزمایش‌های مختلف انجام دادیمaugmentation :. dropout، pooling، weight decay، .schedulerنتیجه: هیچ‌کدوم بهتر نشدن چون مدل از قبل well-regularizedبود.

۴ .با imbalance مقابله کردیم: باBalancedBatchSampler ، recall کلاس‌های نادر رو از ۰,۰۳ به ۰,۷۸ رسوندیم.

۵ .CE و BCE رو مقایسه کردیم: با tuning برابر، BCEبرنده شد = F1 ۰,۶۳۱ vs ۰,۵۹۶

۶ .Transfer Learning رو اجرا کردیم: سه تا استراتژی تست کردیم. بهترین : ResNet-18با fine-tuning که به = F1 ۰,۹۳۲ رسید.

نتایج مدل:

مدل نهایی ResNet-FT روی validation به این نتایج رسید:

macro_F1 = ۰,۹۳۲

accuracy = ۰,۹۴۶

macro_P = ۰,۹۳۶

macro_R = ۰,۹۳۰

چالش‌هایی که داشتیم:

۱ .کلاس :vanet همیشه ضعیف‌ترین کلاس بود~ F1 ۰,۷۹چون فقط ۱۷ تصویر توی validation داشت و با minibus/savari شبیه بود.

۲ .کلاس :kamyunet توی CNNBaseline خیلی ضعیف بود = F1 ۰,۳۰ولی با fine-tuningحل شد = F1 ۰,۹۲

۳ .آمبولانس vs وانت: بعضی آمبولانس‌ها روی شاسی مزدا وانت ساخته شدن. تفاوت فقط توی تجهیزات (چراغ، علائم) هست. این یه چالش دامنه‌ایه، نه اشتباه برچسب.

۴ :Overconfidence .مدل حتی وقتی اشتباه می‌کنه، confidenceبالایی داره.

این برای production خطرناکه.

خروجی نهایی:

یه تابع predict که یه مسیر تصویر می‌گیره و JSON برمی‌گردونه:

predicted_class: "taxi"

confidence: 0.87

probabilities: {...}

needs_review: false

به زبان دیگه :

این پروژه نشون داد که Transfer Learning چقدر می‌تونه مؤثر باشه. با فقط ۴ هزار پارامتر آموزش‌پذیر توی feature extraction تونستیم از CNN با ۱۱ میلیون پارامتر بهتر عمل کنیم. یعنی دانش pretrained از ImageNet خیلی قوی‌تر از آموزش از صفر روی دیتای کوچیکه.

همچنین یاد گرفتیم که:

accuracyتنها کافی نیست توی imbalance ، macro-F1مهم‌تره

مقایسه‌ی عادلانه نیاز به tuning برابر داره ،

confidenceبالا لزوماً به معنای درست بودن نیست،

unseen classes نیاز به OOD detection دارن